

GRAAL: Graph-based Research with Agents for Automatic Labelling

Automatic classification in statistical nomenclatures using an agentic approach

Meilame Tayebjee

meilame.tayebjee@insee.fr

Insee, SSP Lab

Théo Ferry

theo.ferry@insee.fr

Insee, SSP Lab

2 April 2026

Outline

1. **Context.** Challenges of automatic coding
2. **The GRAAL framework.** Multi-agent architecture
3. **Perspectives.** Future developments

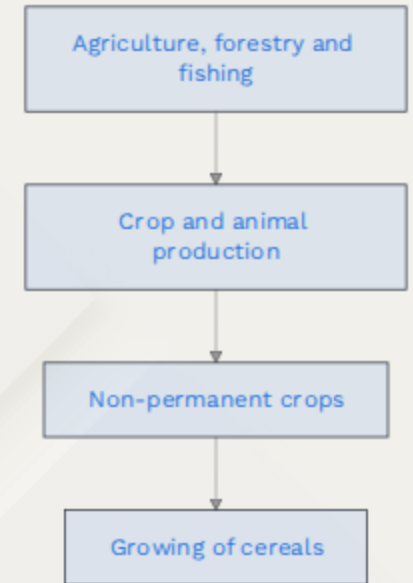
1 Context: the automatic coding use case

NAF : Statistical Nomenclature of Economic Activities

- ~700 final codes
- Hierarchical structure over 5 levels

COICOP : *Classification of Individual Consumption by Purpose*

- ~300 to ~1000 depending on granularity
- Hierarchical structure over 5 levels



Challenges

- Currently: classical supervised learning with *deep learning* models (PyTorch)
 - Frugal, performant and fast, *production-ready*... 😎
 - ... but requires between 100k and 1M labelled data points 🤖

MLOps: facilitated by Onyxia, but still incomplete

- **Data:** S3 storage (Datalab), versioning with MLFlow Datasets ✓
- **Model:** in-house development and distribution via *torchTextClassifiers*, parallelised training with Argo Workflows, model storage with MLFlow ✓
- **Deployment:** FastAPI deployed via containerisation (Docker) ✓
- **Monitoring:** this is where it falls short ✗
 - Requires human investment, not always done
 - **How to monitor the model in production without continuous human annotation?**

The need to move away from labelled data

- What to do when labelled data is unavailable or insufficient?
- What to do when the nomenclature is updated?
 - Existing data needs to be re-coded
- What to do when our labelled data is unreliable?
 - and ideally we want to correct it...

A first zero-shot approach: RAG

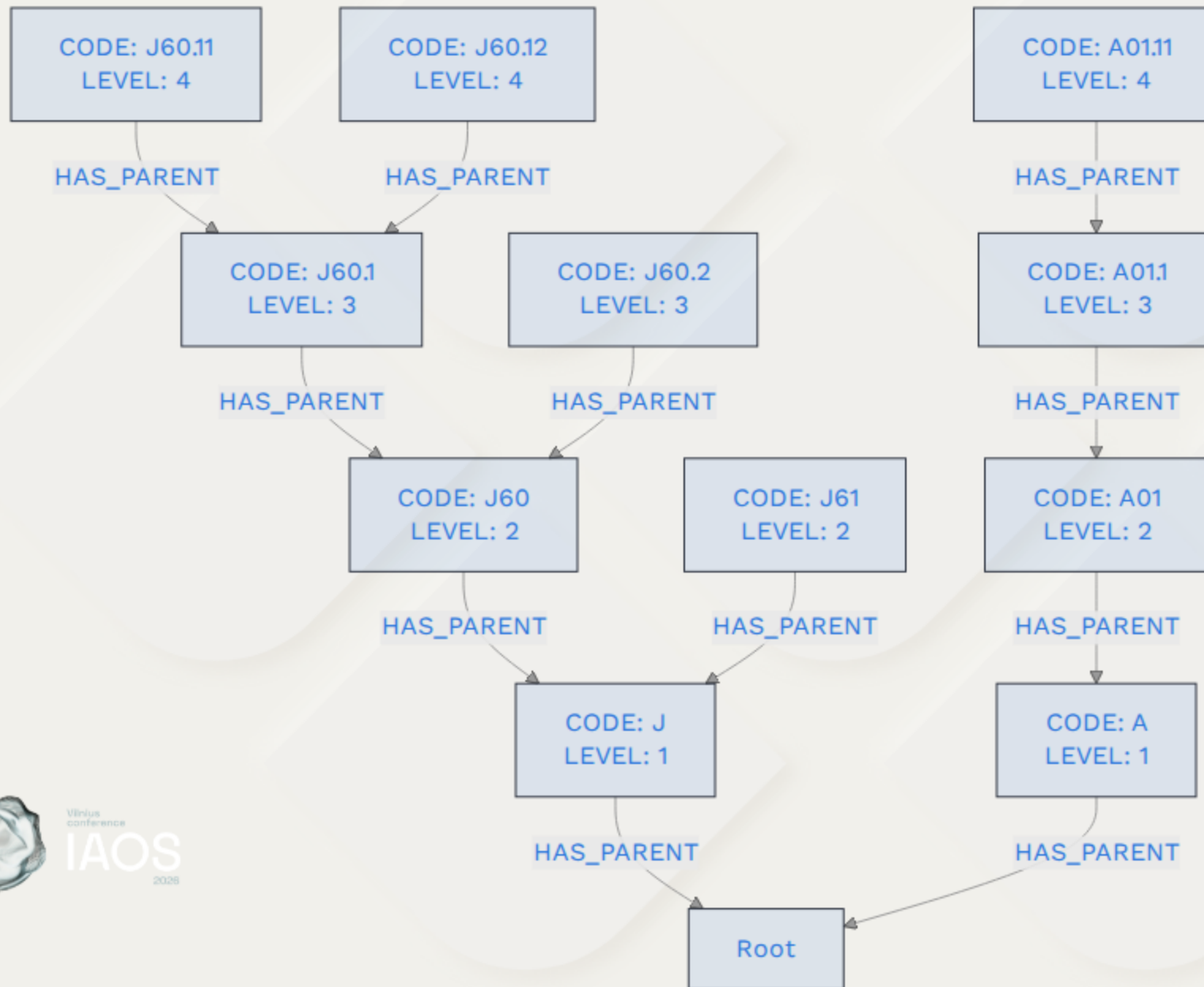
- A first **zero-shot** approach: **Retrieval-Augmented Generation (RAG)**
 - Knowledge base = official descriptions for each code
 - A good first solution, but...
 - dependency on chunking
 - dependency on the embedding model
 - context saturation with descriptions
 - comparing a *description* embedding to a *label* embedding
 - no traceability, no reasoning
 - no consideration of the hierarchy!

Objectives

- No more embeddings
- We want *reasoning*: the LLM can/must justify its choices (*chain of thought*)
 - Rather than *imposing context* as in a RAG, let the model **complete its context dynamically** (= **agent!**)
 - Ideally, with reasoning around the hierarchy

2 The GRAAL framework

Neo4j: a *graph* database



Using the graph as a foundation for agents

- We define a set of tools **shared by all agents**:
 - get_code_information
 - get_current_children
 - get_current_siblings
 - go_to_parent
 - go_to_child
 - etc.
- They all use Neo4j syntax

The agents

- An agent = a prompt, an input/output type, and tools
- The *closers*: **CodeChooser**, **MatchVerifier**
- *Text2Code* (classifier): the **Navigator** inherits from *Text2Code*
- *Code2Text* (synthetic data generator)

MatchVerifier — example implementation

```
1 class MatchVerifier(BaseAgent):
2     def get_instructions(self) -> str:
3         return """You are an agent specialised in verifying
4             the validity of a match between a label
5             and a code in a Neo4j nomenclature graph."""
6
7     def build_prompt(self, input: MatchVerificationInput) -> str:
8         return f"""Verify whether the code matches the activity.
9         Activity: {input.activity}
10        Proposed code: {input.code}
11        Explanation: {input.proposed_explanation}
12        Respond with: validity (bool), confidence (0-1), explanation."""
```

Classification example with *Navigator*

Input

```
1 description = ""  
2 Artisan bakery producing  
3 specialty breads and pastries,  
4 with direct in-store sales  
5 ""
```

Output

```
1 {  
2   "code": "10.71Z",  
3   "label": "Manufacture of bread  
4             and fresh pastry goods",  
5   "confidence": 0.94,  
6   "path": ["C", "10", "10.7", "10.71", "10.71Z"],  
7   "justification": "Artisan manufacturing (C)  
8                     of bakery products (10.7)  
9                     with direct sales."  
10 }
```

Evaluation

- **Evaluating a classifier agent**

- If we trust our test dataset: straightforward evaluation
- **Otherwise: manual evaluation!**
 - Ground truth prediction, classifier prediction, *CodeChooser* and *MatchVerifier* judgements combined
 - Joint evaluation of all components

- **Evaluating synthetic data generation**

- Less straightforward
- Retrain a model on a synthetic dataset and check whether equivalent performance is reached on the same test set

Perspectives

- Requirements met
 - No more *embeddings*
 - Reasoning and traceability
 - **Usable for any nomenclature:** just create a Neo4j database with the descriptions and everything works!
 - GRAAL is just a distributor of tools, prompts and LLM call structures (input/output validation)
- **The main task ahead is evaluation** 

What will we do with it?

- An agentic model is too heavy for production (no GPU yet, inference time too high...)
- However, GRAAL can be used for:
 - **Monitoring the deployed model:** *MatchVerifier* analyses API logs, evaluates the model continuously, calls the *Navigator* if re-coding is needed
 - **Re-coding a database** (same workflow as monitoring)
 - **Synthetic data generation** (*Code2Text*)

Thank you for your attention!

 github.com/InseeFrLab/GRAAL